

Smart Construction Project Resource Planning System

Review 1 Documentation — Problem Understanding, Research & Initial DSA Design

DETAILED PROJECT

DOCUMENTATION

Field	Details
Course	Data Structures and Algorithms - II
Course Code	CCSE0301
Programme / Branch	B.Tech CSE-A
Assignment Type	Individual Assignment / PBL
Project	Smart Construction Project Resource Planning System
Review	Review 1 — Problem Understanding & Initial Progress
Review Coverage	Unit/Module 1 (Trees) and Unit/Module 2 (Graphs)
Current Progress	25%
Status	Research, problem analysis, DSA applicability study and conceptual design; implementation planned/under development

Note: This document is structured as detailed supporting documentation for Review 1. It separates completed analysis and design work from implementation decisions and future development.

Table of Contents

- 1. Introduction
- 2. Project Domain and Context
- 3. Problem Statement
- 4. Problem Understanding and Identification
- 5. Existing Construction Planning and Research Background
- 6. Literature Review
- 7. Research Gap Identified
- 8. Project Objectives
- 9. Stakeholders and User Requirements
- 10. Functional and Data Requirements

11. DSA Concepts Studied, Selection, Implementation and Rejection Analysis
12. Detailed Tree-Based Analysis
13. Detailed Graph-Based Analysis
14. Resource Prioritization and Allocation Method
15. Proposed End-to-End System Workflow
16. Conceptual Data Model
17. Proposed Algorithms and Pseudocode
18. Complexity and Performance Considerations
19. Work Completed During Month 1
20. Evidence and Supporting Material
21. Challenges Faced and How They Were Addressed
22. Individual Contribution
23. Current Status and Progress
24. Plan for Review 2
25. Conclusion
26. References

1. Introduction

Construction projects involve many activities that must be completed in a planned sequence while sharing limited resources such as labour, materials, equipment, time and budget. As project size increases, coordinating these activities and resources becomes more difficult.

The proposed Smart Construction Project Resource Planning System studies this problem from a Data Structures and Algorithms perspective. The objective is not simply to store project records, but to organize project hierarchy, represent activity dependencies, search resource records, prioritize activities and manage activities waiting for resources.

The Review 1 work focuses on understanding the construction resource-planning problem, studying relevant research, identifying a practical project gap, evaluating DSA structures against real operations, and designing the initial system architecture. The implementation status is explicitly separated so that the report does not claim functionality that has not yet been demonstrated.

The documentation therefore addresses four questions:

- Why is construction resource planning a meaningful computing problem?
- What difficulties arise when activities and resources increase?
- Why are particular DSA structures selected or rejected?
- How will the selected structures work together in the proposed system?

2. Project Domain and Context

2.1 Domain

Domain: Construction project resource planning and algorithmic management of activities, dependencies and limited resources.

2.2 Why this domain was selected

Construction planning naturally contains hierarchical information, dependency relationships, resource records and priority decisions. This provides a practical environment in which multiple DSA concepts can be applied without artificially adding unrelated structures.

Entity	Important Attributes	Why It Matters
Project	Project ID, name, location, start date, target completion date, budget	Defines the overall construction work.
Activity	Activity ID, name, duration, status, priority, predecessor activities	Represents individual work that must be planned and scheduled.
Worker	Worker ID, name, skill, daily wage, availability, days worked	Supports labour allocation, attendance and income tracking.
Equipment	Equipment ID, type, availability, operating status	Represents limited machinery and shared equipment.
Material	Material ID, name, quantity, unit, reorder level	Supports inventory and material availability management.
Dependency	Predecessor, successor, relationship type	Defines which activities depend on others.
Resource Allocation	Activity, resource, quantity, allocation status	Connects available resources with project activities.

3. Problem Statement

Construction projects manage many interdependent activities while working with limited labour, materials, equipment, time and budget. Manual or sequential planning can make it difficult to locate resources quickly, identify conflicts, maintain material availability and determine which activities should be processed first.

The project therefore addresses the problem of efficiently organizing construction activities and their dependencies, searching and managing resource records, prioritizing activities, and handling resource constraints as project data grows.

The problem can be expressed as:

- Given a construction project containing activities, dependencies and available resources, identify a feasible execution order.
- Determine whether the resources required by an activity are available.
- Manage competing activities when a resource is shared or temporarily unavailable.
- Track workers, equipment and material records efficiently.
- Identify activities that may be affected when a predecessor is delayed.
- Prioritize available activities according to defined project criteria.

The system focuses on using appropriate data structures and algorithms to make these operations organized, efficient and explainable.

4. Problem Understanding and Identification

4.1 Understanding developed during Review 1

Area	Understanding
Existing problem	Construction resource planning is an established problem, but coordinating activities, dependencies and limited resources remains computationally challenging.
Users affected	Project managers, site engineers, contractors, workers and small construction firms.

Core information	Activities, project phases, dependencies, workers, materials, equipment, availability and priorities.
Core operations	Search, insert, delete, update, traversal, dependency analysis, prioritization and queue management.
Main DSA need	Efficient representation of hierarchy and relationships, fast resource lookup, ordered resource management and priority-based activity processing.
Project direction	Build a transparent DSA-based planning system rather than attempting to reproduce complex commercial construction platforms.

4.2 Problem formulation

The system treats construction planning as a sequence of related operations rather than as one algorithm. First, project activities and resources are represented. Dependencies are then modelled, available activities are identified, priorities are applied, and resources are assigned or placed into waiting queues when unavailable.

```

Project / Activities
    ↓
Project Hierarchy (General Tree)
    ↓
Activity Dependencies (Graph)
    ↓
Dependency Analysis (BFS / DFS / Topological Sort)
    ↓
Resource Search (Hash Table / AVL Tree)
    ↓
Priority Management (Max Heap / Min Heap)
    ↓
Resource Waiting (Queue)
    ↓
Resource Allocation and Monitoring

```

5. Existing Construction Planning and Research Background

Construction resource planning is not a new research problem. Existing studies have examined resource-constrained scheduling, labour planning, material management, simulation, optimization, BIM, IoT and AI-based decision support.

The purpose of reviewing existing work is therefore not to claim that resource planning has been overlooked. Instead, the project uses the literature to understand established challenges and then explores a smaller, transparent implementation based on fundamental DSA concepts.

Aspect	Observed Research Direction	Project Focus
Resource allocation	Optimization and resource-constrained scheduling	Efficient representation and allocation using DSA
Activity dependencies	Scheduling models and dependency constraints	Graph-based dependency model
Labour planning	Labour demand estimation and resource allocation	Worker records, availability and allocation
Material management	Inventory monitoring, waste reduction and procurement planning	Material availability and resource records
Decision support	Simulation, AI/ML and advanced	Simple, explainable algorithmic

	optimization	decision support
Implementation complexity	Many approaches target large or specialized systems	Academic-scale, modular and testable DSA implementation

6. Literature Review

The following research directions were reviewed to understand the construction resource-planning problem and the role of computational methods.

6.1 Resource management at modern construction sites: Bridging the gap between scientific knowledge and industry practice and needs (2024)

Reviews resource management practices and practitioner perspectives. The study highlights gaps between scientific knowledge and construction-site practice and emphasizes better monitoring and data-driven decision making.

Relevance: Supports the need for structured resource information and practical decision support.

6.2 Multi-Objective Resource-Constrained Scheduling in Large and Repetitive Construction Projects (2024)

Studies resource-constrained scheduling as a complex optimization problem involving multiple objectives, resource availability and project delays.

Relevance: Supports the view that activity scheduling becomes difficult when resources and project constraints interact.

6.3 Evaluating how effective project planning influences construction delays by examining planning quality, resource allocation and risk mitigation strategies (2026)

Examines relationships among planning quality, resource allocation, risk mitigation and construction delays using stakeholder data.

Relevance: Supports the importance of planning and resource alignment for schedule predictability.

6.4 Labour multiplier driven approach for budgeting project resources (2024)

Develops labour multiplier profiles using completed public construction projects to support more systematic labour resource budgeting.

Relevance: Supports the inclusion of worker demand, availability and labour-related resource planning.

6.5 Tracing resource flows and reducing environmental impacts during construction: Assessment framework for on-site waste efficiency (2025)

Examines material flows and construction waste, including losses associated with over-ordering and planning issues.

Relevance: Supports including material availability, consumption and waste-related information in resource planning.

7. Research Gap Identified

The literature shows that construction resource planning is commonly addressed through optimization, simulation, BIM, AI/ML, IoT and other advanced approaches. The project does not claim that these methods are absent or ineffective.

Observation	Basis	Project Response
Resource planning involves multiple interacting constraints.	Construction scheduling/resource-	Model activities, resources and dependencies separately and connect them through

	management literature.	algorithms.
Advanced methods can involve substantial modelling and computational complexity.	Observed across optimization, simulation and AI-oriented research.	Explore a simpler, transparent DSA implementation suitable for academic evaluation.
Resource information must be accessed and updated frequently.	Resource-management problem context.	Use Hash Table and AVL Tree for different search/indexing needs.
Activities have dependency relationships.	Scheduling problem.	Use Graph, Adjacency List, BFS, DFS and Topological Sort.
Available activities may have different priorities.	Scheduling/resource allocation requirement.	Use Max Heap and Min Heap for priority-based processing.
The project requires an explainable architecture.	Project design decision.	Give every DSA structure a clear operation and responsibility.

7.1 Core project gap statement

The project explores how fundamental data structures and algorithms can be combined into a transparent construction resource-planning workflow that represents project hierarchy, activity dependencies, resource records and activity priorities in one modular system.

7.2 Important viva clarification

The reviewed papers motivate the construction resource-planning problem and identify practical challenges. They do not prescribe the exact DSA structures used in this project. The selection of General Tree, AVL Tree, Graph, Adjacency List, heaps, Queue and Hash Table is a project-specific engineering decision based on the operations required.

8. Project Objectives

1. Understand the practical problem of construction activity and resource planning.
2. Study existing construction resource-management and scheduling approaches.
3. Identify a project-specific gap suitable for a DSA-focused academic implementation.
4. Represent project phases and sub-activities using a General Tree.
5. Represent activity dependencies using a Graph and Adjacency List.
6. Use BFS, DFS and Topological Sort for dependency analysis and activity ordering.
7. Use AVL Tree and Hash Table for efficient resource record management.
8. Use Max Heap and Min Heap for priority-based activity processing.
9. Use a Queue to manage activities waiting for shared resources.
10. Analyse the time and space complexity of the implemented structures and algorithms.

9. Stakeholders and User Requirements

Stakeholder	Need	Expected System Support
Project Manager	Plan activities and allocate limited resources	Activity hierarchy, dependency analysis, priority management and resource allocation
Site Engineer	Monitor activity status and dependencies	Activity records, graph traversal and feasible execution order
Contractor	Manage labour, equipment and materials	Resource records, availability and allocation information
Worker	View work-related information	Worker record, attendance, days worked and income calculation
Small Construction Firm	Use a simple planning system	Modular and explainable DSA-

	without complex enterprise tools	based workflow
--	----------------------------------	----------------

9.1 User workflow

11. Create or select a construction project.
12. Define project phases and activities.
13. Add activity dependencies.
14. Register workers, equipment and materials.
15. Check available resources for an activity.
16. Prioritize activities according to the selected rule.
17. Allocate available resources or place unavailable activities in a queue.
18. Update activity and resource status as work progresses.

10. Functional and Data Requirements

10.1 Functional requirements

- Create and organize project phases and sub-activities.
- Add and maintain activity dependencies.
- Search and update worker, equipment and material records.
- Check resource availability before activity allocation.
- Prioritize activities using defined priority criteria.
- Generate a dependency-valid activity sequence.
- Explore downstream activities affected by a delayed activity.
- Maintain a waiting queue for activities whose required resources are unavailable.
- Maintain worker attendance, days worked and income information.
- Measure algorithmic operation time and complexity during testing.

10.2 Proposed activity record

```
Activity {
    id
    name
    phase
    duration
    status
    priority
    deadline
    requiredResources[]
    predecessorActivities[]
}
```

10.3 Proposed worker record

```
Worker {
    id
    name
    skill
    dailyWage
    daysWorked
    availability
    attendance[]
}
```

10.4 Proposed equipment record

```
Equipment {
    id
    name
    type
    availability
    status
}
```

10.5 Proposed material record

```
Material {  
    id  
    name  
    quantity  
    unit  
    reorderLevel  
    availability  
}
```

11. DSA Concepts Studied, Selection, Implementation and Rejection Analysis

The project evaluates each DSA concept against a concrete warehouse-like construction operation: representing hierarchy, representing dependencies, searching resources, maintaining ordered records, prioritizing activities, exploring dependency paths and managing waiting activities.

The implementation status below reflects the current project design and code scope specified for this project. A structure marked implemented should appear in the corresponding project module and demonstration.

11.1 DSA Selection Philosophy

The project follows a problem-first approach: identify the real construction operation, determine what the system needs to do, compare suitable structures, and then select the structure that provides the required behaviour with appropriate complexity.

11.2 Data processed by the DSA structures

The main data consists of project phases, activities, dependency relationships, workers, equipment, materials, resource availability and activity priorities. The same project can therefore be represented through different structures for different operations.

11.3 General Tree — Implemented

Used for project hierarchy. A phase can contain multiple activities, so a General Tree is more natural than a Binary Tree. It supports hierarchical display, phase organization and bottom-up/top-down processing.

11.4 Binary Tree — Not Implemented as Core

A Binary Tree allows at most two children and has no inherent search ordering. It does not match the project's multi-child hierarchy and does not provide a direct advantage over the General Tree for the main project structure.

11.5 BST — Not Implemented as Core

A BST provides ordered search, but it can become skewed and degrade to $O(n)$. The project uses AVL where balanced ordered search is required, so a separate BST implementation is not necessary in the core system.

11.6 AVL Tree — Implemented

Used for ordered worker/resource records. AVL automatically maintains balance and guarantees $O(\log n)$ search, insertion and deletion, making performance predictable as records grow.

11.7 Binary Heap / Max Heap — Implemented

The heap structure supports priority-based activity management. Max Heap is used when the system needs quick access to the highest-priority activity.

11.8 Min Heap — Implemented

Used when a smaller value should receive priority, such as an earliest deadline, shortest duration or earliest availability criterion. Its use depends on the scheduling rule being evaluated.

11.9 Tree Traversal — Implemented

Preorder, postorder and inorder traversal support project hierarchy processing and verification of tree-based records. Each complete traversal is $O(n)$.

11.10 Threaded Binary Tree — Not Implemented

Its main benefit is specialized traversal with reduced dependence on recursion or an auxiliary stack. The current system does not have a traversal bottleneck that justifies this additional complexity.

11.11 Graph — Implemented

Used for activity dependencies. Unlike a hierarchy, dependencies can connect an activity with multiple predecessors or successors, making a directed graph appropriate.

11.12 Adjacency List — Implemented

Used as the primary graph representation because the expected dependency graph is sparse and traversal is a major operation.

11.13 Adjacency Matrix — Not Implemented

Although it provides $O(1)$ direct edge checking, it requires $O(V^2)$ space. The Adjacency List is more suitable when only a limited number of activity relationships exist.

11.14 BFS — Implemented

Used for level-by-level exploration of activity dependencies and for identifying activities across dependency levels.

11.15 DFS — Implemented

Used to follow dependency chains and explore downstream activities that may be affected by a delay.

11.16 Topological Sort — Implemented

Used to generate an activity order that respects predecessor relationships, provided the dependency graph is acyclic.

11.17 Queue — Implemented

Used for activities waiting for a shared resource. FIFO processing provides a clear waiting-line model.

11.18 Hash Table — Implemented

Used for fast ID-based lookup of workers, equipment and materials. Expected average lookup is $O(1)$.

11.19 Linked List — Not Implemented as Core

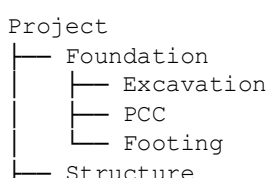
A Linked List is useful when frequent dynamic insertion/deletion is the main requirement. The current system does not have a core operation that requires this enough to justify another structure.

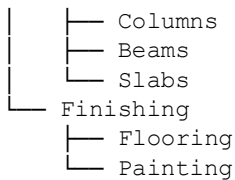
12. Detailed Tree-Based Analysis

12.1 Why a tree is considered

Construction projects contain hierarchical information: a project contains phases, phases contain activities, and activities can be grouped into sub-activities. A tree is therefore appropriate for the project hierarchy, while different tree variants can support ordered resource indexing.

12.2 General Tree





12.3 AVL Tree operations

Operation	Purpose	Complexity
Search	Find worker/resource by ordered key	$O(\log n)$
Insert	Add a resource record	$O(\log n)$
Delete	Remove a resource record	$O(\log n)$
Inorder traversal	Display ordered records	$O(n)$

12.4 Tree traversal

Traversal	Purpose in Project	Complexity
Preorder	Display project hierarchy from parent to children	$O(n)$
Inorder	Display AVL/BST records in sorted key order	$O(n)$
Postorder	Bottom-up phase/activity processing	$O(n)$
Level-order	Breadth-wise inspection of a tree	$O(n)$

13. Detailed Graph-Based Analysis

13.1 Why a graph is necessary

Construction activities can have multiple dependency relationships. For example, a finishing activity may require both structural work and wall work to be completed. A graph represents these cross-connections without forcing the project into a single parent-child hierarchy.

13.2 Proposed graph model

```

Excavation —PRECEDES—> Foundation
Foundation —PRECEDES—> Columns
Foundation —PRECEDES—> Walls
Columns —PRECEDES—> Beams
Walls —PRECEDES—> Plastering
Beams —PRECEDES—> Slab
Plastering —PRECEDES—> Finishing
Slab —PRECEDES—> Finishing
  
```

13.3 Graph components

Component	Representation
Vertex	Construction activity
Directed Edge	Dependency between activities
Weight (optional)	Duration, delay impact, priority or another project metric
Adjacency List	Collection of directly dependent activities for each vertex

13.4 Adjacency List vs Adjacency Matrix

Aspect	Adjacency List	Adjacency Matrix
Space	$O(V + E)$	$O(V^2)$
Best suited for	Sparse graphs	Dense graphs
Edge check	$O(\text{degree})$	$O(1)$
Traversal	$O(V + E)$	$O(V^2)$
Project fit	Preferred for activity dependencies	Not preferred as core representation

13.5 BFS and DFS

Algorithm	Project Use	Complexity
BFS	Explore dependency levels and nearby activities	$O(V + E)$
DFS	Trace downstream dependency paths	$O(V + E)$
Topological Sort	Generate dependency-valid activity order	$O(V + E)$

14. Resource Prioritization and Allocation Method

14.1 Why priority management is required

Several activities may be ready at the same time, while resources such as cranes, skilled workers or equipment may be limited. The system therefore needs a way to select activities according to an explicit priority rule rather than relying only on input order.

14.2 Max Heap

The Max Heap stores activities according to a priority value. The activity with the highest priority remains at the root and can be selected efficiently.

Activity	Priority
Foundation	90
Columns	80
Beams	70
Painting	30

Max Heap → Foundation is available at the root.

14.3 Min Heap

A Min Heap is used when the scheduling rule gives higher priority to a smaller value, such as an earliest deadline or shortest duration.

14.4 Queue for unavailable resources

If a required resource is unavailable, the activity can be placed in a waiting queue. When the resource becomes available, queued activities can be reconsidered according to the project's processing policy.

14.5 Resource allocation logic

Select an activity that satisfies its dependencies

↓

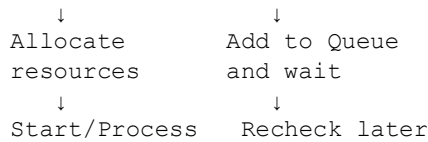
Check required resources

↓

Resources available?

YES

NO



15. Proposed End-to-End System Workflow

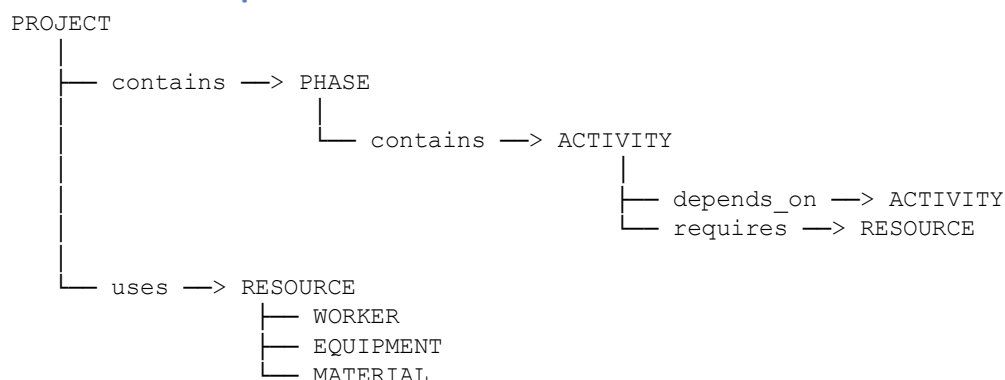
19. Create the construction project and its phases.
20. Add activities under the appropriate project phase.
21. Create dependency relationships between activities.
22. Build the activity graph using an adjacency list.
23. Use Topological Sort to identify a dependency-valid sequence.
24. Check resource records using Hash Table and AVL Tree.
25. Identify activities whose required resources are currently available.
26. Assign a priority and process available activities using Max Heap or Min Heap.
27. Place activities with unavailable shared resources into a Queue.
28. Use BFS or DFS when dependency exploration or delay-impact analysis is required.
29. Update activity and resource status as work progresses.
30. Record worker attendance, days worked and income where applicable.

15.1 Why the stages are separated

Stage	Reason
Project hierarchy	Keeps phases and sub-activities organized.
Dependency modelling	Separates relationship information from hierarchy.
Dependency ordering	Ensures prerequisite activities are respected.
Resource retrieval	Avoids repeatedly scanning all resource records.
Priority management	Provides an explicit rule for choosing among available activities.
Waiting management	Handles activities whose resources are temporarily unavailable.
Monitoring	Keeps activity and resource information current.

16. Conceptual Data Model

16.1 Relationship model



16.2 Example record set

Activity	Duration	Priority	Required Resource	Status
----------	----------	----------	-------------------	--------

Excavation	3 days	80	Excavator + Workers	Ready
Foundation	5 days	90	Workers + Concrete	Waiting
Columns	4 days	85	Workers + Crane	Blocked
Beams	4 days	70	Workers + Crane	Pending
Painting	3 days	30	Painters + Paint	Pending

16.3 Example worker record

Worker ID	Name	Daily Wage	Days Worked	Total Income
1056	Rahul	₹700	22	₹15,400
1057	Amit	₹750	20	₹15,000
1058	Ravi	₹650	24	₹15,600

Total Income = Daily Wage × Days Worked

17. Proposed Algorithms and Pseudocode

17.1 Activity search using AVL

```
searchActivity(root, key):
    if root == NULL:
        return NOT_FOUND
    if key == root.key:
        return root
    if key < root.key:
        return searchActivity(root.left, key)
    else:
        return searchActivity(root.right, key)
```

17.2 BFS dependency traversal

```
BFS(start):
    create queue Q
    mark start as visited
    push start into Q
    while Q is not empty:
        u = remove front of Q
        process u
        for each v in adjacencyList[u]:
            if v is not visited:
                mark v visited
                push v into Q
```

17.3 DFS delay-impact traversal

```
DFS(u):
    mark u as visited
    process u
    for each v in adjacencyList[u]:
        if v is not visited:
            DFS(v)
```

17.4 Topological Sort

```
TopologicalSort(graph):
    calculate indegree of every activity
    add all activities with indegree 0 to a queue
    while queue is not empty:
        u = remove front
        add u to ordering
```

```

    for each v adjacent to u:
        decrease indegree[v]
        if indegree[v] becomes 0:
            add v to queue
    if ordering contains all activities:
        return ordering
    else:
        report dependency cycle

```

17.5 Max Heap priority processing

```

for each ready activity:
    maxHeap.insert(priority, activity)

while maxHeap is not empty:
    activity = maxHeap.extractMax()
    process or allocate resources to activity

```

17.6 Resource lookup

```

findResource(id):
    index = hash(id)
    search the corresponding hash-table bucket
    return resource record if found

```

18. Complexity and Performance Considerations

Implemented Structure / Algorithm	Main Operation	Complexity	Why It Matters
General Tree	Traversal	$O(n)$	Every project node may need to be processed.
General Tree	Search	$O(n)$	No ordering is assumed in the hierarchy.
AVL Tree	Search	$O(\log n)$	Provides predictable ordered lookup.
AVL Tree	Insert/Delete	$O(\log n)$	Maintains efficient updates as records grow.
Hash Table	Search	$O(1)$ average	Fast ID-based resource lookup.
Hash Table	Insert/Delete	$O(1)$ average	Efficient resource record updates.
Max Heap	Get Maximum	$O(1)$	Immediate access to highest priority.
Max Heap	Insert	$O(\log n)$	Maintains heap property.
Max Heap	Delete Maximum	$O(\log n)$	Efficiently removes highest priority.
Min Heap	Get Minimum	$O(1)$	Immediate access to minimum criterion.
Min Heap	Insert/Delete Minimum	$O(\log n)$	Efficient minimum-based scheduling.
Queue	Enqueue/Dequeue	$O(1)$	Efficient waiting-resource management.

Adjacency List	Add Edge	$O(1)$ typical	Efficient dependency insertion.
BFS	Traversal	$O(V + E)$	Efficient sparse-graph exploration.
DFS	Traversal	$O(V + E)$	Efficient dependency-chain exploration.
Topological Sort	Generate Order	$O(V + E)$	Efficient dependency-valid sequencing.
Tree Traversal	Pre/In/Postorder	$O(n)$	Processes every relevant tree node.

18.1 Space considerations

- General Tree uses space proportional to the number of project nodes.
- AVL Tree stores one node per indexed record, with logarithmic height.
- Hash Table requires storage for records and table buckets; performance depends on the load factor and collision handling.
- Adjacency List uses $O(V + E)$ space.
- Adjacency Matrix would require $O(V^2)$ space and is therefore not part of the core implementation.

18.2 Evaluation planned for later reviews

- Compare resource-search performance as the number of records increases.
- Measure AVL insertion, deletion and search operations.
- Measure BFS, DFS and Topological Sort as the number of activities and dependencies increases.
- Evaluate heap-based priority processing with different numbers of ready activities.
- Measure queue processing under different resource-availability scenarios.
- Compare practical memory use of adjacency-list representation with a matrix baseline where appropriate.

19. Work Completed During Month 1

- Finalized the project title and construction resource-planning direction.
- Defined the problem of coordinating activities, dependencies and limited resources.
- Identified project stakeholders and core information requirements.
- Studied existing construction resource-management and scheduling approaches.
- Reviewed recent research relevant to resource management, scheduling, labour planning and material management.
- Identified a project-specific gap for a transparent DSA-based implementation.
- Studied DSA-II Unit 1 concepts including Binary Tree, BST, AVL, Threaded Binary Tree, traversal and heap structures.
- Studied DSA-II Unit 2 concepts including graph representation, adjacency structures and graph traversal.
- Evaluated which DSA concepts are genuinely useful for the project and which are not required.
- Defined the General Tree project hierarchy and Graph dependency model.
- Mapped resource-management operations to AVL Tree, Hash Table and Queue.
- Defined priority-management logic using Max Heap and Min Heap.
- Prepared the implementation and complexity-analysis direction for the next review.

19.1 What has not been claimed as completed

- Advanced optimization or AI-based scheduling is not claimed as part of the current implementation.
- Final performance results are not claimed until controlled testing is completed.
- Real-world construction data is not claimed unless it is actually incorporated and documented.
- Research findings are not presented as proof that the proposed DSA system outperforms advanced industrial systems.

20. Evidence and Supporting Material

20.1 Evidence placement

The project evidence should be organized around the implemented DSA components and their outputs.

Evidence	What It Should Demonstrate
General Tree output	Project phases and sub-activities are stored and traversed correctly.
AVL Tree output	Resource records can be inserted, searched, updated and maintained in balanced order.
Hash Table output	Resource records can be retrieved using IDs.
Graph output	Activity dependencies are represented correctly.
BFS/DFS output	Dependency exploration produces the expected visited sequence.
Topological Sort output	Generated activity order respects dependencies.
Heap output	Activities are selected according to the implemented priority rule.
Queue output	Activities waiting for unavailable resources are maintained correctly.

21. Challenges Faced and How They Were Addressed

Challenge	How It Was Addressed
Choosing DSA structures without artificially using every syllabus topic	Each structure was mapped to a concrete construction operation before selection.
Representing hierarchy and dependencies separately	General Tree was assigned to hierarchy and Graph to dependencies.
Handling resource records efficiently	Hash Table was selected for direct ID lookup and AVL for ordered records.
Managing different priority rules	Max Heap and Min Heap were separated according to whether larger or smaller values have priority.
Handling unavailable shared resources	Queue was introduced for activities waiting for resources.
Keeping the project academically explainable	Every structure is documented with its purpose, alternative and complexity.

22. Individual Contribution

- Defined the Smart Construction Project Resource Planning System problem direction.
- Studied the construction resource-planning context and existing research.
- Analysed the applicability of DSA-II concepts to the project.
- Designed the project hierarchy using a General Tree.
- Designed the activity dependency model using Graph and Adjacency List.
- Mapped BFS, DFS and Topological Sort to dependency operations.
- Mapped AVL Tree and Hash Table to resource management.
- Mapped Max Heap, Min Heap and Queue to priority and waiting-resource management.
- Prepared complexity analysis and implementation scope.
- Prepared the proposed workflow and data model for subsequent development.

23. Current Status and Progress

Area	Status
Problem understanding	Completed
Domain and context analysis	Completed
Research/literature study	Completed
Research gap identification	Completed
DSA applicability analysis	Completed
General Tree design/implementation	Implemented
Graph and Adjacency List	Implemented
BFS / DFS / Topological Sort	Implemented
AVL Tree	Implemented
Hash Table	Implemented
Max Heap / Min Heap	Implemented
Queue	Implemented
System-wide integration	Under development / to be demonstrated
Performance evaluation	Planned
Final documentation	In progress

The implementation status should be updated after each review so that the report remains consistent with the actual project code.

24. Plan for Review 2

31. Finalize the data model for projects, activities and resources.
32. Integrate the General Tree with project and phase management.
33. Complete and test AVL Tree and Hash Table resource operations.
34. Complete Graph and Adjacency List integration.
35. Test BFS, DFS and Topological Sort with multiple dependency scenarios.
36. Integrate Max Heap and Min Heap into activity-priority processing.
37. Integrate Queue-based waiting-resource management.
38. Connect worker attendance and income calculations to worker records.
39. Create test cases for normal, boundary and conflicting-resource scenarios.
40. Measure time and space behaviour as input size increases.
41. Prepare screenshots and output evidence for each implemented DSA component.

24.1 Review 2 implementation target

The next review should demonstrate a working pipeline in which project activities are organized, dependencies are represented and processed, resources are searched and allocated, priorities are handled, and waiting activities are managed using the selected DSA structures.

25. Conclusion

Review 1 establishes the problem, research background and DSA design for the Smart Construction Project Resource Planning System. The project treats construction resource planning as a combination of hierarchy management, dependency analysis, resource retrieval, priority management and waiting-resource handling.

The selected DSA structures are not included merely to satisfy a syllabus checklist. General Tree, Graph, Adjacency List, BFS, DFS, Topological Sort, AVL Tree, Hash Table, Max Heap, Min Heap and Queue each have a defined responsibility within the system. Binary Tree, BST, Adjacency Matrix, Linked List and Threaded

Binary Tree were evaluated but excluded from the core implementation where their advantages did not match the project's main requirements.

The next stage is to integrate the modules, test them with increasing data sizes and document the measured performance. The final goal is a practical and explainable academic system that demonstrates how DSA concepts can be applied to a real construction resource-planning problem.

26. References

1. Nielsen, L. H., Lindhard, S. M., & Kanafani, K. (2024). Resource management at modern construction sites: Bridging the gap between scientific knowledge and industry practice and needs. *Journal of Environmental Management*, 366, 121835. DOI: 10.1016/j.jenvman.2024.121835.
2. Lazari, V., Chassiakos, A., & Karatzas, S. (2024). Multi-Objective Resource-Constrained Scheduling in Large and Repetitive Construction Projects. *Algorithms*, 17(8), 351. DOI: 10.3390/a17080351.
3. Acharya, R., Yadav, M., & Thakur, V. (2026). Evaluating how effective project planning influences construction delays by examining planning quality, resource allocation and risk mitigation strategies. *Engineering, Construction and Architectural Management*. DOI: 10.1108/ECAM-10-2025-1606.
4. Cheung, S.-W. T., Siu, M.-F. F., & Chan, P.-C. A. (2024). Labour multiplier driven approach for budgeting project resources. *Developments in the Built Environment*, 19, 100496. DOI: 10.1016/j.dibe.2024.100496.
5. Hasselsteen, L., Damgaard, A., Otovic, A. P., Birgisdóttir, H., & Kanafani, K. (2025). Tracing resource flows and reducing environmental impacts during construction: Assessment framework for on-site waste efficiency. *Journal of Cleaner Production*, 521, 146243. DOI: 10.1016/j.jclepro.2025.146243.

DSA references may be added separately according to the textbooks or course material prescribed by the institution.